

Compiler-Fused Training Pipeline (CFTP)

Compiler-Native Gradient Accumulation and Packing-Aware Kernel Synthesis for NeuralScript

Authors: Brandon Wiemer

Date: April 2026

Status: Research Proposal

Abstract

We propose two compiler-native training optimizations for NeuralScript that exploit static knowledge unavailable to runtime frameworks: (1) **Fused Accumulation-Step Elimination (FASE)**, which uses source-level AD and memory planning to interleave gradient accumulation, optimizer steps, and parameter updates into a single backward pass — eliminating gradient buffers entirely while remaining mathematically compatible with gradient accumulation (something PyTorch's `register_post_accumulate_grad_hook` explicitly cannot do); and (2) **Pack-Compiled Attention (PCA)**, which bakes sequence packing document boundaries directly into the attention kernel's PTX at compile time, eliminating the runtime attention mask tensor and its associated HBM traffic. Together, these techniques address the two training acceleration methods not covered by our previous WRGA and CSHA research.

Part I: Fused Accumulation-Step Elimination (FASE)

1. The Gradient Accumulation Memory Problem

Gradient accumulation simulates larger effective batch sizes by running N micro-batches, accumulating gradients, then applying one optimizer step. The standard pipeline:

```
for micro_batch in 0..N:
    forward(micro_batch)           # compute activations
    backward()                     # compute gradients, ADD to gradient
    buffer                         # gradient buffer persists across all
                                # N micro-batches
optimizer.step()                  # read gradient buffer, update
parameters
optimizer.zero_grad()             # clear gradient buffer
```

The problem: The gradient buffer has the same size as the model parameters and persists across all N micro-batches. For NSLCoder-50M (48.8M parameters, FP32): that's 195MB of gradient storage that sits allocated for the entire accumulation window.

PyTorch offers `register_post_accumulate_grad_hook` to fuse the optimizer step into the backward pass, eliminating gradient buffers by applying updates immediately. **But this is incompatible with gradient accumulation**, as documented by PyTorch itself and the optimizer library: "features which rely on persistent gradients like gradient accumulation won't work." The fundamental tension: you can fuse backward+optimizer OR you can accumulate gradients, but not both.

2. FASE: The Compiler-Native Solution

NSL resolves this tension because the compiler has complete static knowledge of the accumulation count, the optimizer algorithm, and the backward computation graph.

2.1 Core Insight: Accumulation-Aware Optimizer Fusion

When the user writes:

```
ns!

train(model=m, epochs=1, grad_clip=1.0):
    optimizer: AdamW(lr=0.001, weight_decay=0.01)
    ...
```

with `grad_accumulation=4`, the compiler knows at compile time:

- The exact accumulation count $N=4$
- The optimizer algorithm (AdamW) and all its hyperparameters
- The complete backward graph (via source AD or tape recording structure)
- The memory layout of all parameters and optimizer states

FASE uses this to generate a fused backward that maintains a **running optimizer state update** rather than a gradient buffer:

2.2 Mathematical Foundation

For AdamW with accumulation count N , the standard computation per parameter θ is:

```
g_acc = (1/N)  $\sum_i$  g_i           # accumulated gradient (average)
m_t =  $\beta_1$  m_{t-1} + (1- $\beta_1$ ) g_acc  # first moment
v_t =  $\beta_2$  v_{t-1} + (1- $\beta_2$ ) g_acc2 # second moment
 $\theta_t = \theta_{t-1} - lr \times (\hat{m}_t / (\hat{v}_t + \epsilon) + wd \times \theta_{t-1})$ 
```

FASE rewrites this as an incremental update. Instead of accumulating g into a buffer, we maintain *running moment accumulators* that are updated per micro-batch:

```
# After micro-batch  $i$  ( $i = 0..N-1$ ):  
m_partial_i = m_partial_{i-1} + (1/N) ×  $g_i$            # running sum of scaled gradients  
v_partial_i = v_partial_{i-1} + (1/N²) ×  $g_i^2$          # running sum of scaled squared  
gradients  
  
# After micro-batch  $N-1$  (final):  
# m_partial_{N-1} = (1/N)  $\sum g_i = g_{acc}$              ← mathematically identical  
# v_partial needs correction:  
# v_partial_{N-1} = (1/N²)  $\sum g_i^2 \neq ((1/N) \sum g_i)^2$  ← not identical (variance  
decomposition)
```

The v_t correction: The second moment $v_t = \beta_2 v_{t-1} + (1-\beta_2) g_{acc}^2$ uses g_{acc}^2 , which is the square of the *mean* gradient, not the mean of the *squared* gradients. FASE handles this by:

Option A (exact): Store $m_{partial}$ per-parameter (same size as gradient buffer — no savings). This is what PyTorch's hook approach does and why it's incompatible.

Option B (FASE-approximate): Use the running squared-gradient mean as an approximation: $v_t \approx \beta_2 v_{t-1} + (1-\beta_2) \times (1/N) \sum g_i^2$. This is the *batch-variance-aware* second moment, which actually provides a richer signal than the standard computation. Research on gradient noise scale shows this is often superior for training stability.

Option C (FASE-deferred, the novel approach): Accumulate only the first moment incrementally, and defer the second moment and parameter update to after the final micro-batch. The key: the first moment accumulation $m_{partial} += (1/N) \times g_i$ requires only one scalar multiply-add per parameter per micro-batch — no gradient buffer needed. After the final micro-batch, one standard optimizer step completes using $m_{partial}$ as g_{acc} .

2.3 FASE-Deferred: How the Compiler Implements It

```
FASE-Deferred Backward (compiled, per micro-batch i):
```

```
for each parameter  $\theta_k$  in reverse topological order:
     $g_k = \text{compute\_gradient}(\theta_k)$       # standard backward for this parameter

    if  $i < N-1$ : # not final micro-batch
         $m_{\text{partial}_k} += (1/N) * g_k$       # accumulate into first-moment buffer (IN-PLACE)
        #  $g_k$  is immediately freed — never stored in a gradient buffer

    else: # final micro-batch ( $i == N-1$ )
         $m_{\text{partial}_k} += (1/N) * g_k$       # final accumulation
         $g_{\text{acc}_k} = m_{\text{partial}_k}$         #  $m_{\text{partial}}$  IS the accumulated gradient

    # Full optimizer step (fused into backward epilogue):
     $m_k = \beta_1 * m_k + (1-\beta_1) * g_{\text{acc}_k}$ 
     $v_k = \beta_2 * v_k + (1-\beta_2) * g_{\text{acc}_k}^2$ 
     $\theta_k -= \text{lr} * (\hat{m}_k / (\sqrt{\hat{v}_k} + \epsilon) + \text{wd} * \theta_k)$ 

     $m_{\text{partial}_k} = 0$                     # reset for next accumulation window
```

Memory comparison (NSLCoder-50M, 48.8M params, FP32):

Method	Gradient Buffer	Optimizer State	Total Extra
Standard accumulation	195 MB	390 MB (m, v)	585 MB
PyTorch fused (no accum)	0 MB	390 MB	390 MB
FASE-Deferred	0 MB*	390 MB + 195 MB (m_{partial})	585 MB

Wait — *the total is the same?* Yes, because `$m_{\text{partial}}$` is the same size as the gradient buffer. But FASE's advantage isn't memory savings — it's **peak memory timing**.

2.4 The Real Win: Peak Memory Scheduling

The standard pipeline has this memory timeline:

STANDARD:

```
micro-batch 0: [params] + [activations0] + [grads0 building]
micro-batch 1: [params] + [activations1] + [grads0 + grads1 building]
...
micro-batch N-1: [params] + [activationsn] + [full grad buffer]
optimizer step: [params] + [full grad buffer] + [optimizer state] ← PEAK
```

FASE has this timeline:

```
FASE-Deferred:
  micro-batch 0: [params] + [activations0] + [m_partial building] ← m_partial
grows layer-by-layer
  (after backward0): [params] + [m_partial] ← activations freed, m_partial same
size as grads
  micro-batch 1: [params] + [activations1] + [m_partial]
  ...
  micro-batch N-1 (backward):
    for each layer (reverse):
      compute gk → fuse optimizer step → free gk → free activationk
    # At each point, only ONE layer's gradient exists
```

The critical difference: In FASE, gradients exist for only ONE layer at a time during the fused backward-optimizer step on the final micro-batch. The compiler's memory planner (M36) schedules this so that each gradient is computed, used for the optimizer step, and freed before the next layer's gradient is computed.

Actual peak memory savings (NSLCoder-50M):

Method	Peak Memory	When Peak Occurs
Standard	Params + Full Grads + Optimizer + Last Activations = ~1.2 GB	Optimizer step
FASE	Params + m_partial + Optimizer + 1 Layer Grad = ~0.8 GB	Mid-backward of final micro-batch

The savings come from never holding ALL gradients simultaneously — only one layer's gradient at a time during the fused step.

2.5 Gradient Clipping Compatibility

Gradient clipping requires the global gradient norm, which normally requires all gradients to be computed before clipping. FASE handles this with a two-phase final micro-batch:

```
Phase 1: Backward pass – compute gradients, accumulate m_partial, compute running
norm2
Phase 2: Clip + optimizer step – one more pass over m_partial with clip factor
applied
```

This requires reading m_partial twice on the final micro-batch (one extra HBM read of the accumulated gradient), but still avoids the full gradient buffer.

The compiler generates this two-phase approach automatically when it detects `grad_clip` in the train block config.

2.6 Why PyTorch Cannot Do This

1. **No static accumulation count.** PyTorch doesn't know N at "compile time" (there is no compile time). The `register_post_accumulate_grad_hook` fires after every backward, with no way to know whether this is micro-batch 0 or N-1.
 2. **No per-layer optimizer fusion.** PyTorch's optimizer operates on all parameters simultaneously (`optimizer.step()`). Fusing per-layer during backward requires the compiler to inline the optimizer's update rule into each layer's backward epilogue.
 3. **No memory scheduling.** PyTorch's memory allocator uses a dynamic caching scheme. It cannot plan ahead to reuse gradient memory for the next layer's gradient because it doesn't know the allocation-free pattern in advance.
 4. **No source-AD integration.** The fused backward+accumulation+optimizer is a single generated function, not a composition of runtime hooks. The compiler can optimize across all three concerns holistically.
-

Part II: Pack-Compiled Attention (PCA)

3. The Sequence Packing Runtime Tax

Sequence packing concatenates multiple short documents into one long sequence to eliminate padding waste. The challenge: the model must not attend across document boundaries.

The standard solution is a **packing-aware attention mask** — a tensor that encodes which tokens can attend to which. For packed sequences, this mask has a block-diagonal structure:

Documents: [A A A | B B | C C C C] (packed into one sequence of length 9)

Attention Mask:

```
1 1 1 0 0 0 0 0 0
1 1 1 0 0 0 0 0 0
1 1 1 0 0 0 0 0 0
0 0 0 1 1 0 0 0 0
0 0 0 1 1 0 0 0 0
0 0 0 0 0 1 1 1 1
0 0 0 0 0 1 1 1 1
0 0 0 0 0 1 1 1 1
0 0 0 0 0 1 1 1 1
```

Problems with runtime masking:

1. **Memory:** The mask is $S \times S$ per batch element. For `seq_len=2048`: $4\text{M entries} \times 4 \text{ bytes} = 16\text{MB}$ per batch element. For `batch_size=32`: **512MB** just for the mask.
2. **HBM traffic:** The mask must be loaded into shared memory alongside Q, K, V during FlashAttention. This increases HBM reads by the mask size per tile iteration.
3. **Computational waste:** Even with the mask, FlashAttention still computes attention scores for masked positions and then applies `-inf`. The QK^T matmul wastes FLOPs on cross-document entries that will be zeroed.
4. **Runtime mask construction:** Building the block-diagonal mask from document lengths requires Python-level logic per batch, adding CPU overhead to the data pipeline.

PyTorch's FlexAttention addresses this somewhat via compiled `block_mask` functions, but still operates at runtime with dynamic dispatch.

4. PCA: Compile-Time Packing-Aware Kernels

PCA takes a fundamentally different approach: the document boundary structure is baked into the attention kernel's control flow at compile time.

4.1 Static Packing Configuration

NSL's `DataLoader` with `SequencePacker` produces packed sequences with a known packing layout. The compiler extracts this layout from the `dataset` block:

```
ns!

dataset train_data("corpus"):
    source = MemoryMapped("data/tokens.bin", dtype=int32)
    sequence_length = 1024
    packing = true
    pack_separator = tokenizer.eos_token_id
```

The compiler knows:

- Maximum sequence length (1024)
- Packing is enabled
- Document boundaries are marked by `eos_token_id`

4.2 The PCA Kernel: Three Strategies

Strategy 1: Segment-ID Kernel (General Case)

Instead of an $S \times S$ mask, PCA passes a compact **segment ID vector** — one integer per token indicating which document it belongs to. The attention kernel checks `segment_id[q_pos] == segment_id[k_pos]` instead of loading a mask entry:

```
Segment IDs: [0, 0, 0, 1, 1, 2, 2, 2, 2] ← 9 ints vs 81-entry mask
```

```
Kernel modification (inside FlashAttention KV-tile loop):
```

```
    for each q_pos, k_pos in tile:
        if segment_id[q_pos] != segment_id[k_pos]:
            score = -inf
```

HBM savings: Segment IDs are S integers (4KB for $\text{seq}=1024$) vs $S \times S$ mask entries (4MB).

1000× reduction.

The segment IDs are loaded into shared memory once and reused across all KV tiles — negligible overhead.

NSL implementation: The compiler detects `(packing = true)` in the dataset config and generates a FlashAttention variant with segment-ID-aware masking. The segment IDs are computed during data loading (inside the DataLoader's zero-copy mmap path) with no Python overhead.

Strategy 2: Document-Boundary Tile Skipping (Compile-Time Optimization)

When the compiler knows the sequence length and typical document length distribution (from dataset statistics or user annotation), it can generate a kernel that skips entire KV tiles known to be fully cross-document:

```
Packed sequence: [Doc A: 400 tokens | Doc B: 300 tokens | Doc C: 324 tokens]
Block size: 128 tokens
```

```
Tile map (which Q-tiles attend to which KV-tiles):
```

```
    Q-tiles 0-2 (Doc A tokens 0-383): attend to KV-tiles 0-3 only
```

```
    Q-tile 3 (Doc A tokens 384-399 + Doc B tokens 0-15): attend to KV-tiles 0-4
(boundary tile)
```

```
    Q-tiles 3-5 (Doc B): attend to KV-tiles 3-5 only
```

```
    ...
```

For fully within-document tiles (no boundary crossing), the causal mask is applied without any segment check. Only boundary tiles need segment-ID checking. The compiler generates a tile-skip map as a compile-time constant embedded in the kernel.

FLOP savings: For a sequence of 1024 tokens packed with 3 documents of ~340 tokens each, approximately 67% of QK^T entries are cross-document and can be skipped. With tile-level skipping, this becomes ~55% of tile-pairs skipped entirely (no matmul computed).

Strategy 3: Multi-Sequence Per-CTA (Maximum Efficiency)

For short documents (average length $\ll$ max sequence length), PCA generates a kernel where each CTA processes one complete document rather than one Q-tile of a packed sequence:


```
Standard FlashAttention grid: (num_q_tiles batch × heads, 1)
PCA grid: (num_documents, batch × heads, 1)
```

Each CTA:

1. Read document start/end from a compact document offset table
2. Load only this document's Q, K, V tiles
3. Standard FlashAttention loop within document boundaries
4. Write output to the correct position in the packed output

Advantage: Zero wasted compute on cross-document entries. No mask needed at all. Each CTA processes exactly the tokens it should.

Disadvantage: Uneven CTA workloads (documents have different lengths). The compiler's cost model determines whether the load imbalance penalty exceeds the cross-document waste penalty.

4.3 Position ID Fusion

Packed sequences require per-document position IDs (positions reset at document boundaries). In PyTorch, this is a separate tensor constructed at runtime. PCA fuses position ID computation into the RoPE kernel:

```
Standard: position_ids[i] = i - document_start[segment_id[i]]
          → separate tensor, separate HBM allocation
```

```
PCA: RoPE kernel reads segment_ids and document_starts from shared memory,
     computes position offset on-the-fly during rotation
     → no position_id tensor ever exists
```

Combined with CSHA Level 1 (boundary fusion), RoPE is already fused into the projection epilogue. PCA adds document-aware position reset to this fused kernel — zero additional cost.

4.4 Compile-Time Loss Masking

When packing, the loss should only be computed on non-padding tokens. NSL's existing `cross_entropy` implementation with `valid_mask` handles this, but PCA can go further:

The fused linear cross-entropy kernel (LM head matmul + softmax + cross-entropy in one kernel) can skip entire document-separator tokens at the kernel level, avoiding the LM head matmul for tokens whose loss will be masked. The compiler determines which token positions are separators from the packing configuration and generates a kernel that skips them.

FLOP savings for loss computation: For a packed sequence with 3 documents of ~340 tokens and 2 separator tokens, this saves $2/1024 = 0.2\%$ of LM head FLOPs — tiny. But the real win is avoiding materializing logits for separator tokens ($2 \times \text{vocab_size} \times 4 \text{ bytes} = 384\text{KB}$ for $\text{vocab}=49152$), which reduces peak memory at the loss computation boundary.

4.5 Backward Pass Integration

PCA's document boundary information flows through the backward pass via source AD:

1. **Attention backward:** The segment-ID-aware forward kernel saves its tile-skip map. The backward kernel reuses this map to skip the same cross-document tile pairs, saving the same FLOPs.
2. **RoPE backward:** The document-aware position computation in the forward is automatically differentiated by source AD. The backward correctly applies per-document position gradients without a separate position_id tensor.
3. **Loss backward:** Cross-entropy backward only computes gradients for non-separator tokens (matching the forward skip).

4.6 Why PyTorch Cannot Do This

1. **No compile-time packing layout.** PyTorch constructs masks at runtime in the DataLoader's collate function. The FlashAttention kernel receives the mask as a runtime tensor — it cannot specialize its control flow based on mask structure.
 2. **FlexAttention limitations.** PyTorch's FlexAttention can compile a `block_mask` function, but it still evaluates the mask at runtime per-tile. It cannot skip entire tile-pairs because it doesn't know the document layout until the batch is constructed. PCA knows the layout structure (not the specific documents, but the packing pattern) at compile time.
 3. **No cross-kernel packing awareness.** Even with FlexAttention's mask, PyTorch still needs a separate position_id tensor and separate loss masking. PCA fuses all three (attention mask + position IDs + loss mask) into a single compile-time packing configuration that flows through the entire training pipeline.
 4. **No tile-skip maps.** PyTorch cannot generate tile-skip maps because the mask is a dense tensor. PCA's segment-ID approach enables block-level skip decisions in the kernel.
-

5. FASE + PCA Integration

FASE and PCA compose naturally because they operate at orthogonal levels of the training pipeline:

```
Compiled Training Step (FASE + PCA, accumulation=4):
```

```
for micro_batch in 0..4:
    # Forward pass with PCA:
    x = dataloader.next()                # packed sequence + segment_ids (from
mmap)
    logits = model.forward(x)            # PCA attention kernels (segment-
aware, no mask)
    loss = cross_entropy(logits, labels)  # fused CE with separator skipping

    # Backward pass with FASE:
    if micro_batch < 3:
        backward_accumulate(loss)        # compute grads, add 1/4 × g to
m_partial, free g
    else:
        backward_fused_step(loss)        # compute grads, complete
accumulation, optimizer step

                                           # per-layer: g → m_partial → clip →
AdamW step → free
```

Combined benefits (NSLCoder-50M, batch=32, seq=1024, accumulation=4):

Optimization	Memory Saved	Compute Saved
PCA: No attention mask tensor	512 MB	—
PCA: Tile skipping (~55% cross-doc)	—	~50% of attention FLOPs
PCA: No position_id tensor	128 KB	—
PCA: Fused RoPE position reset	—	1 kernel launch
FASE: No gradient buffer (peak)	~200 MB peak	—
FASE: Per-layer fused optimizer	—	N-1 gradient HBM round-trips
Total	~712 MB	~50% attention + optimizer fusion

6. NSL Language Surface

6.1 FASE

FASE is transparent — the compiler applies it automatically when gradient accumulation is configured:

```
ns!
```

```
# FASE activates automatically: compiler knows accumulation=4 and optimizer=AdamW
train(model=m, epochs=1, grad_clip=1.0):
    optimizer: AdamW(lr=0.0003, weight_decay=0.1)
    step(batch):
        let logits = m.forward_train(batch.input_ids, true)
        let loss = cross_entropy(logits, batch.labels)
```

The `grad_accumulation` parameter in the train block config triggers FASE compilation. No user annotation needed.

6.2 PCA

PCA activates when the compiler detects `packing = true` in a dataset definition that feeds the train block:

```
ns!
```

```
dataset train_data("corpus"):
    source = MemoryMapped("data/tokens.bin", dtype=int32)
    sequence_length = 1024
    packing = true
    pack_separator = tokenizer.eos_token_id

train(model=m, epochs=1):
    data:
        source = train_data
        batch_size = 32
    step(batch):
        # batch.input_ids is packed, batch.segment_ids is auto-generated
        # PCA kernels used automatically – no attention mask constructed
        let logits = m.forward_train(batch.input_ids, true)
        let loss = cross_entropy(logits, batch.labels)
```

6.3 Compile-Time Diagnostics

```
bash

$ nsl check --training-report pretrain.nsl --target rtx5070ti

=== Training Pipeline Report ===
Target: NVIDIA RTX 5070 Ti (16GB VRAM)
Model: NSLCoder-50M (48.8M params, FP32)

Gradient Accumulation: 4 micro-batches
  FASE mode: Deferred (first-moment accumulation)
  Gradient buffer: ELIMINATED (peak reduced by ~200MB)
  Optimizer fusion: per-layer AdamW fused into final micro-batch backward
  Gradient clipping: two-phase (norm computation + clipped step)

Sequence Packing: ENABLED
  PCA strategy: Segment-ID kernel (general case)
  Mask memory saved: 512 MB (no S×S mask tensor)
  Position ID: fused into RoPE kernel
  Tile skipping: estimated 55% cross-document tiles skipped
  Loss masking: fused into CE kernel

Estimated peak VRAM: 4.2 GB (vs 5.1 GB without FASE+PCA)
Estimated training throughput: +35% vs standard pipeline
```

7. Implementation Plan

7.1 FASE

Component	Location	LOC (est.)	Description
FASE pass	<code>nsl-codegen/src/fase.rs</code>	600	Detects accumulation config, rewrites backward
Per-layer optimizer inline	<code>nsl-codegen/src/fase_optimizer.rs</code>	400	Generates per-layer AdamW/SGD/Lion step code
Gradient clip two-phase	<code>nsl-codegen/src/fase_clip.rs</code>	200	Norm accumulation + clip pass generation
Memory schedule	<code>nsl-codegen/src/fase_memory.rs</code>	300	Integrates with M36 memory planner for peak reduction

Dependencies: source_ad.rs, memory_planner.rs, stmt.rs (train block codegen)

7.2 PCA

Component	Location	LOC (est.)	Description
Packing detection	<code>nsl-codegen/src/pca_detect.rs</code>	200	Analyzes dataset/train block for packing config
Segment-ID kernel variant	<code>nsl-codegen/src/flash_attention.rs</code>	400	New FA variant with segment-ID masking
Tile-skip map generator	<code>nsl-codegen/src/pca_tileskip.rs</code>	300	Compile-time tile-skip map from packing stats
RoPE position fusion	<code>nsl-codegen/src/pca_rope.rs</code>	200	Document-aware position reset in fused RoPE
Segment-ID DataLoader	<code>nsl-runtime/src/dataloader.rs</code>	150	Emit segment_ids alongside packed tokens

Dependencies: flash_attention.rs, dataloader.rs, cost_model.rs

8. Conclusion

FASE and PCA complete the compiler-native training optimization suite alongside WRGA (PEFT) and CSHA (attention fusion). Together, the four techniques address every major training acceleration method through the lens of compile-time optimization:

Training Technique	Standard Approach	NSL Compiler-Native Approach
Flash Attention	Pre-compiled CUDA library	CSHA : compiler fuses across attention boundaries
Gradient Accumulation	Runtime hooks (incompatible with fusion)	FASE : compile-time fused accumulation+optimizer
Sequence Packing	Runtime S×S attention mask	PCA : compile-time segment-ID kernel synthesis
MQA/GQA	Runtime expand+contiguous	CSHA : zero-copy stride patterns
torch.compile	Runtime tracing + Inductor	All four : AOT whole-program compilation
Parameter-Efficient FT	Runtime adapter management	WRGA : compile-time dead gradient elimination

The unifying insight across all four papers: **decisions that frameworks make at runtime using dynamic dispatch, NSL makes at compile time using static analysis** — producing faster, smaller, and more memory-efficient training programs.

References

1. Pudipeddi, B., et al. "Training Large Neural Networks with Constant Memory using a New Execution Algorithm." arXiv 2104.00237, 2021.
2. PyTorch. "How to save memory by fusing the optimizer step into the backward pass." PyTorch Tutorials, 2023.
3. Korthikanti, V., et al. "Reducing Activation Recomputation in Large Transformer Models." MLSys 2023.
4. Dao, T. "FlashAttention-2: Faster Attention with Better Parallelism and Work Partitioning." ICLR 2024.
5. Hsu, P., et al. "Liger-Kernel: Efficient Triton Kernels for LLM Training." 2024.
6. Krell, M., et al. "Efficient Sequence Packing without Cross-contamination." ACL 2022.